Cahier des charges — CineHub

API REST de gestion de cinéma (films, séances, réservations)

Formation	Développement Backend / Node.js
Type de projet	Évaluation / TP de reprise
Durée indicative	2 à 4 séances
Livrable	API REST fonctionnelle + script SQL + collection Postman

---

1. Contexte et objectif

Vous devez concevoir et développer une **API REST** permettant de gérer un cinéma en ligne : consulter les films à l'affiche, réserver une séance, consulter ses réservations, et administrer le catalogue (films, salles, séances).

L'API doit être sécurisée : seul un utilisateur authentifié (via un token JWT) peut accéder aux ressources protégées. Certaines actions (création / modification / suppression de films, salles et séances) sont réservées aux **administrateurs**.

Objectifs pédagogiques

À la fin de ce projet, vous serez capable de :

- Structurer une API Node.js / Express en couches (Routes, Controllers, Models, Middleware, Config)
- Mettre en place une authentification avec inscription, connexion, hachage de mot de passe (bcrypt) et JWT
- Protéger des routes via un middleware de vérification de token
- Distinguer les rôles **utilisateur** et **administrateur**
- Interagir avec une base MySQL (CRUD, jointures, clés étrangères, contraintes d'unicité)
- Exposer une API testable via Postman

---

2. Stack technique imposée

Élément	Technologie
Runtime	Node.js (modules ES6 : `"type": "module"`)
Framework	Express
Base de données	MySQL
Connexion BDD	mysql2/promise
Authentification	jsonwebtoken + bcrypt
Variables d'environnement	dotenv
Relance auto (dev)	nodemon

Dépendances attendues

```
{
  "dependencies": {
    "bcrypt": "^6.0.0",
    "dotenv": "^18.0.5",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
    "mysql2": "^3.24.5"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

---

3. Architecture du projet

L'arborescence suivante est imposée :

```
cinehub/
├── Config/
│   └── db.js                    # Connexion MySQL
├── Controllers/
│   ├── auth.controller.js
│   ├── users.controller.js
│   ├── films.controller.js
│   ├── rooms.controller.js
│   ├── screenings.controller.js
│   └── reservations.controller.js
├── Middlewares/
│   ├── auth.middleware.js       # Vérification du token JWT
│   └── admin.middleware.js      # Vérification du rôle admin
├── Models/
│   ├── users.model.js
│   ├── films.model.js
│   ├── rooms.model.js
│   ├── screenings.model.js
│   └── reservations.model.js
├── Routes/
│   ├── auth.route.js
│   ├── users.route.js
│   ├── films.route.js
│   ├── rooms.route.js
│   ├── screenings.route.js
│   └── reservations.route.js
├── .env
├── .gitignore
├── bdd.sql
├── index.js
└── package.json
```

Responsabilités des couches

Couche	Rôle
Routes	Déclarent les endpoints et appellent le controller
Controllers	Valident les entrées, gèrent les codes HTTP, appellent le model
Models	Contiennent uniquement les requêtes SQL
Middlewares	Interceptent la requête (JWT, rôle admin) avant le controller
Config	Centralise la connexion à la base de données

---

4. Base de données

4.1 Création

Créer une base nommée `cinehub` et un utilisateur MySQL dédié avec les droits `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

4.2 Schéma

Table `users`

Colonne	Type	Contraintes
us_id	INT	PK, AUTO_INCREMENT
us_username	VARCHAR(255)	NOT NULL, UNIQUE
us_password	TEXT	NOT NULL
us_email	VARCHAR(255)	NOT NULL, UNIQUE
us_role	ENUM('user', 'admin')	NOT NULL, DEFAULT `'user'`

> *À l'inscription via `/auth/register`, le rôle doit toujours être `'user'`.*
> *Un administrateur ne peut être créé que via le script SQL (données de seed) ou via une route admin (bonus).*

Table `films`

Colonne	Type	Contraintes
fi_id	INT	PK, AUTO_INCREMENT
fi_title	VARCHAR(255)	NOT NULL
fi_description	TEXT	
fi_duration	INT	NOT NULL (durée en minutes)
fi_genre	VARCHAR(100)	
fi_release_date	DATE	

fi_is_showing	TINYINT(1)	NOT NULL, DEFAULT 1

Table `rooms` (salles)

Colonne	Type	Contraintes
ro_id	INT	PK, AUTO_INCREMENT
ro_name	VARCHAR(100)	NOT NULL, UNIQUE
ro_capacity	INT	NOT NULL

Table `screenings` (séances)

Colonne	Type	Contraintes
sc_id	INT	PK, AUTO_INCREMENT
sc_film	INT	NOT NULL, FK → films(fi_id)
sc_room	INT	NOT NULL, FK → rooms(ro_id)
sc_datetime	DATETIME	NOT NULL
sc_price	DECIMAL(6,2)	NOT NULL

Table `reservations`

Colonne	Type	Contraintes
re_id	INT	PK, AUTO_INCREMENT
re_user	INT	NOT NULL, FK → users(us_id)
re_screening	INT	NOT NULL, FK → screenings(sc_id)
re_seats	INT	NOT NULL, DEFAULT 1
re_created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Contrainte recommandée : `UNIQUE (re_user, re_screening)` — un utilisateur ne peut réserver qu'une fois la même séance (il peut éventuellement modifier le nombre de places via PATCH, bonus).

4.3 Script SQL attendu

Fournir un fichier `bdd.sql` contenant :

1. La création de la base
2. La création des 5 tables (avec contraintes)
3. La création de l'utilisateur MySQL + `GRANT` + `FLUSH PRIVILEGES`
4. Au moins **1 compte admin** de démonstration (mot de passe déjà hashé bcrypt, ou documenté clairement dans le README pour le générer)
5. Quelques films, salles et séances de seed pour faciliter les tests

---

5. Variables d'environnement

Fichier `.env` (à ne jamais committer) :

```
SERVER_PORT=3000
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=cine_user
DB_PASSWORD=********
DB_NAME=cinehub
JWT_SECRET=une_cle_secrete_longue
```

Le `.gitignore` doit au minimum exclure : `node_modules/` , `.env` .

---

6. Authentification et sécurité

6.1 Inscription — `POST /auth/register`

Corps attendu : `{ "username", "email", "password" }`

- Le mot de passe doit être haché avec **bcrypt** avant insertion
- Le rôle enregistré est forcément `'user'`
- Réponse : **201** + résultat de l'insertion (sans renvoyer le hash du mot de passe)
- En cas d'erreur : **500**
- Champs manquants : **403**

6.2 Connexion — `POST /auth/login`

Corps attendu : `{ "username", "password" }`

- Vérifier que le username et le mot de passe sont fournis
- Récupérer l'utilisateur par username
- Comparer le mot de passe avec `bcrypt.compareSync`
- Si OK : générer un JWT contenant les infos utiles (`us_id` , `us_username` , `us_email` , `us_role`), expiration **1 heure**
- Réponse : `{ "message", "token" }`
- Identifiants incorrects : **403**
- Erreur serveur : **500**

Les routes `/auth/*` restent publiques (pas de middleware).

6.3 Middleware `checkToken`

Créer un middleware qui :

1. Lit le header `Authorization`
2. Extrait le token au format `Bearer <token>`
3. Si absent → **401** `{ "error": "Unauthorized" }`
4. Vérifie le token avec `jwt.verify` et `JWT_SECRET`
5. Si invalide → **403** `{ "error": "Token incorrect" }`
6. Si valide → stocke l'utilisateur décodé dans `req.user` puis appelle `next()`

6.4 Middleware `checkAdmin`

Créer un middleware (à placer **après** `checkToken`) qui :

1. Vérifie que `req.user.us_role === 'admin'`
2. Sinon → **403** `{ "error": "Accès réservé aux administrateurs" }`
3. Si OK → `next()`

6.5 Protection des routes

Dans `index.js` :

Préfixe	Auth	Admin
`/auth`	Non	Non
`/`	Non	Non
`/films` (lecture)	Oui	Non
`/films` (écriture)	Oui	Oui
`/rooms` (lecture)	Oui	Non
`/rooms` (écriture)	Oui	Oui
`/screenings` (lecture)	Oui	Non
`/screenings` (écriture)	Oui	Oui
`/reservations`	Oui	Selon endpoint
`/users`	Oui	Selon endpoint

---

7. Spécification des endpoints

7.1 Santé de l'API

Méthode	URL	Auth	Description
GET	`/`	Non	Retourne `{ "status": "OK" }`

7.2 Auth (`/auth`)

Méthode	URL	Auth	Description
POST	`/auth/register`	Non	Inscription (rôle `user`)
POST	`/auth/login`	Non	Connexion → retourne un JWT

7.3 Utilisateurs (`/users`) — protégées

Méthode	URL	Qui	Description
GET	`/users`	Admin	Liste tous les utilisateurs

GET	`/users/:id`	Connecté	Détail d'un utilisateur (sans mot de passe)
GET	`/users/me`	Connecté	Profil de l'utilisateur connecté (`req.user`)
PATCH	`/users/:id`	Connecté (soi) ou Admin	Modifier username / email
DELETE	`/users/:id`	Admin	Supprimer un utilisateur

7.4 Films (`/films`)

Méthode	URL	Qui	Description
GET	`/films`	Connecté	Liste des films
GET	`/films/showing`	Connecté	Films à l'affiche (`fi_is_showing = 1`)
GET	`/films/:id`	Connecté	Détail d'un film + ses prochaines séances
POST	`/films`	Admin	Créer un film
PATCH	`/films/:id`	Admin	Modifier un film (champs partiels)
DELETE	`/films/:id`	Admin	Supprimer un film

Création (`POST /films`)

Corps attendu :

```
{
  "title": "Inception",
  "description": "Un voleur s'infiltre dans les rêves.",
  "duration": 148,
  "genre": "Science-fiction",
  "release_date": "2010-07-21",
  "is_showing": true
}
```

Règles :

- `title` et `duration` sont obligatoires → sinon **403**
- Succès → **201**

Modification (`PATCH /films/:id`)

- Film inexistant → **404**
- Mise à jour partielle : ne modifier que les champs fournis
- Succès → message de confirmation + objet mis à jour

Détail (`GET /films/:id`)

Doit renvoyer le film **et** la liste de ses séances à venir, avec le nom de la salle (jointure).

7.5 Salles (`/rooms`)

Méthode	URL	Qui	Description

GET	`/rooms`	Connecté	Liste des salles
GET	`/rooms/:id`	Connecté	Détail d'une salle
POST	`/rooms`	Admin	Créer une salle
PATCH	`/rooms/:id`	Admin	Modifier une salle
DELETE	`/rooms/:id`	Admin	Supprimer une salle

Création (`POST /rooms`)

```
{
  "name": "Salle 1",
  "capacity": 120
}
```

- `name` et `capacity` obligatoires → sinon **403**
- Succès → **201**

7.6 Séances (`/screenings`)

Méthode	URL	Qui	Description
GET	`/screenings`	Connecté	Liste des séances (avec titre du film + nom de la salle)
GET	`/screenings/film/:filmID`	Connecté	Séances d'un film
GET	`/screenings/:id`	Connecté	Détail d'une séance
POST	`/screenings`	Admin	Créer une séance
PATCH	`/screenings/:id`	Admin	Modifier une séance
DELETE	`/screenings/:id`	Admin	Supprimer une séance

Création (`POST /screenings`)

```
{
  "film": 1,
  "room": 2,
  "datetime": "2026-12-01 20:30:00",
  "price": 9.50
}
```

- `film` , `room` , `datetime` et `price` obligatoires → sinon **403**
- Succès → **201**

7.7 Réservations (`/reservations`)

Méthode	URL	Qui	Description
GET	`/reservations`	Admin	Toutes les réservations

GET	`/reservations/me`	Connecté	Mes séances réservées
GET	`/reservations/screening/:screeningID`	Admin	Réservations d'une séance
POST	`/reservations`	Connecté	Réserver une séance
DELETE	`/reservations/:id`	Connecté (sa résa) ou Admin	Annuler une réservation

Création (`POST /reservations`)

```
{
  "screening": 3,
  "seats": 2
}
```

Règles :

- L'utilisateur réservant est pris depuis `req.user` (pas depuis le body)
- `screening` est obligatoire → sinon **403**
- `seats` doit être un entier ≥ 1 (défaut : 1)
- Vérifier que la séance existe → sinon **404**
- Vérifier qu'il reste assez de places :
  `places déjà réservées + seats ≤ capacité de la salle`
  sinon → **403** `{ "error": "Plus assez de places disponibles" }`
- Succès → **201**

Mes réservations (`GET /reservations/me`)

Doit renvoyer pour chaque réservation :

- id de la réservation, nombre de places
- titre du film
- date/heure de la séance
- nom de la salle
- prix

---

8. Gestion des erreurs

Situation	Code HTTP
Succès lecture / update / delete	200
Création réussie	201
Token absent	401
Token invalide / identifiants incorrects / champs obligatoires manquants / pas admin / plus de places	403
Ressource introuvable	404

Erreur serveur / BDD	500

Toutes les réponses d'erreur doivent être au format JSON :

```
{ "error": "Message explicite" }
```

Chaque controller doit utiliser un bloc `try / catch`.

---

9. Point d'entrée `index.js`

Le fichier doit :

1. Charger dotenv
2. Créer l'app Express
3. Activer `express.json()`
4. Monter les routes (avec `checkToken` / `checkAdmin` selon les cas)
5. Exposer `GET /` avec `{ status: "OK" }`
6. Écouter sur `process.env.SERVER_PORT`
7. Afficher en console l'URL locale (`http://127.0.0.1:PORT`)

---

10. Livrables attendus

1. Code source complet selon l'architecture imposée
2. `bdd.sql` : création BDD + tables + utilisateur MySQL + données de seed (admin, films, salles, séances)
3. `.env.example` (sans secrets) listant les variables nécessaires
4. Collection Postman couvrant au minimum :
    - Register / Login (user + admin)
    - Films à l'affiche + détail film
    - CRUD Films / Rooms / Screenings (en tant qu'admin)
    - Réservation + mes réservations + annulation
    - Appels sans token → 401
    - Appels user sur route admin → 403
5. `README.md` court : installation, lancement, comptes de test, exemples d'appels

---

11. Bonus (facultatif)

- Ne jamais renvoyer le mot de passe hashé dans les réponses
- Soft delete sur les films (`fi_is_showing = 0` plutôt que DELETE)
- Empêcher la réservation d'une séance déjà passée
- Route admin pour promouvoir un utilisateur (`PATCH /users/:id/role`)
- Validation plus poussée des emails / dates / prix
- Envoi d'un e-mail de confirmation de réservation (nodemailer)
- Cascade SQL documentée à la suppression d'un film / d'une séance

---

12. Scénario de démonstration suggéré

1. Importer `bdd.sql` (admin seed + films + salles + séances)
2. Créer un utilisateur classique via `POST /auth/register`

3. Se connecter en user → récupérer le token
4. Lister les films à l'affiche (`GET /films/showing`)
5. Consulter un film et ses séances (`GET /films/:id`)
6. Réserver une séance (`POST /reservations`)
7. Consulter ses réservations (`GET /reservations/me`)
8. Se connecter avec le compte **admin**
9. Créer un nouveau film, une salle, une séance
10. Lister toutes les réservations (`GET /reservations`)
11. Appeler une route protégée **sans token** → vérifier le **401**
12. Appeler `POST /films` avec un token **user** → vérifier le **403**
13. Annuler sa réservation (`DELETE /reservations/:id`)

---

13. Contraintes et consignes

- **Interdit** : stocker les mots de passe en clair
- **Interdit** : hardcoder les identifiants BDD ou le secret JWT dans le code
- **Interdit** : mettre toute la logique SQL dans les controllers
- **Interdit** : permettre à un user de s'auto-promouvoir admin via `/auth/register`
- **Obligatoire** : requêtes préparées (`?`) pour éviter les injections SQL
- **Obligatoire** : répondre toujours en JSON
- **Obligatoire** : l'utilisateur d'une réservation vient du token (`req.user`), pas du body
- Le projet doit démarrer avec `npm start` (script nodemon)

Document destiné aux étudiants — à réaliser de manière autonome à partir de ce cahier des charges.